

QLoRA Distillation for Algorithmic Code Classification and Generation

Pranav Bhat P (231IT049)

04-08-2026

1 Introduction and The Original System

Recent work on reasoning distillation for code has focused on transferring chain-of-thought (CoT) traces from large reasoning models into smaller, non-reasoning student LLMs. The baseline methodology followed here is *The Valley of Code Reasoning* (He et al., 2025), which distills competitive-programming reasoning traces from a large teacher model directly into 7B–8B student models (Qwen2.5-7B-Instruct, Llama3.1-8B-Instruct). In this framework, a raw problem statement is passed to the teacher, which produces a rationale and a solution; the student is then fine-tuned on these (rationale, code) pairs. The paper’s key finding is a non-monotonic *valley*: downstream pass-rate first drops, then rises sharper-than-log-linearly, as distillation data increases.

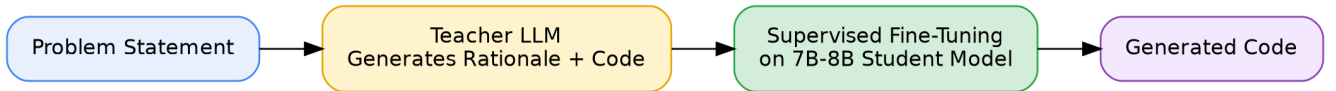


Figure 1: Conceptual diagram of the existing (baseline) system

While this approach demonstrates that reasoning-trace distillation scales for code, it operates as an undifferentiated, single-signal pipeline that offers no structural guidance to the student and has only been validated at large model scale.

2 Observed Demerits of the Baseline Architecture

The baseline architecture, despite its scaling insight, has three limitations that restrict its applicability to resource-constrained and more interpretable settings:

- **No Algorithmic-Category Conditioning:** The student must simultaneously infer *which* algorithmic strategy applies and *how* to implement it, from a single undifferentiated loss signal. There is no intermediate abstraction (e.g., an explicit algorithm tag) to decouple problem understanding from code synthesis.
- **Unvalidated at Consumer-GPU Scale:** All reported experiments use 7B–8B parameter students trained with standard fine-tuning. The paper offers no evidence on whether the “valley” phenomenon persists, shifts, or disappears for sub-2B models trained under QLoRA on constrained hardware (e.g. a single 6GB consumer GPU or free-tier Colab/Kaggle T4).
- **Correctness Filtering Ignores Rationale Validity:** Data subsets are filtered only by final-solution correctness, not by whether the intermediate rationale is a genuinely valid explanation. A trace that reaches the right answer through flawed or mismatched reasoning is still retained as gold supervision, risking transfer of spurious reasoning patterns to the student.

3 Design of the Improved System: The Tag-Conditioned QLoRA Pipeline

To address these gaps, the proposed improvement restructures the single-stage distillation pipeline into a **two-stage, tag-conditioned architecture** trained entirely under QLoRA on consumer-grade hardware. Instead of mapping problem statements directly to code, an explicit algorithm-tag prediction stage is introduced ahead of generation, and teacher rationales are filtered for tag-consistency before being used as supervision.

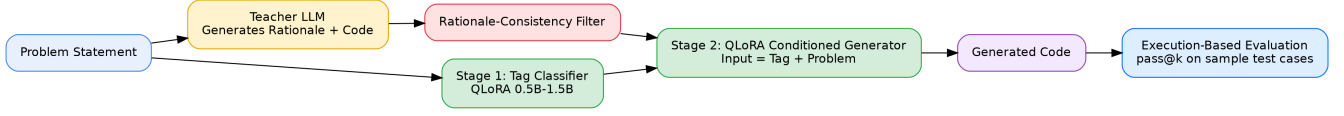


Figure 2: Conceptual diagram of the improved system

This new pipeline consists of three primary components:

- **Stage 1 – Tag Classifier:** A QLoRA-fine-tuned small model (0.5B–1.5B) that predicts the algorithm category (e.g. DP, graphs, greedy) from the problem statement.
- **Rationale-Consistency Filter:** A validation step that discards teacher (rationale, code) pairs whose stated reasoning does not match the ground-truth tag, ensuring the student is only supervised on causally valid traces.
- **Stage 2 – Conditioned Generator:** A QLoRA-fine-tuned code model (e.g. Qwen2.5-Coder-1.5B) that generates code conditioned on both the problem statement *and* the predicted tag, replacing the baseline’s single undifferentiated input.

4 Implementation Workflow

The end-to-end workflow required to realise the improved system is as follows:

1. **Dataset Construction:** Scrape Codeforces problems (statement, tags, difficulty) restricted to easy/medium, single-dominant-tag problems for tractable scope.
2. **Teacher Trace Generation:** Query a large teacher model via API to produce (rationale, solution) pairs for each problem.
3. **Rationale-Consistency Filtering:** Discard pairs where the teacher’s stated reasoning does not align with the problem’s ground-truth tag.
4. **Stage 1 Training:** QLoRA-fine-tune the tag classifier on Kaggle/Colab (T4, free tier), validating with accuracy and F1-macro.
5. **Stage 2 Training:** QLoRA-fine-tune the conditioned generator on (predicted-tag + problem) $\rightarrow$ (rationale + code), sweeping model size and data quantity to test whether tag-conditioning shifts the baseline’s valley curve.
6. **Execution-Based Evaluation:** Run generated code against Codeforces sample test cases in a sandboxed subprocess judge to compute pass@k, compared against an unconditioned ablation trained on the same data without the tag-conditioning signal.